

ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ИРКУТСКОЙ ОБЛАСТИ
«АНГАРСКИЙ ТЕХНИКУМ СТРОИТЕЛЬНЫХ ТЕХНОЛОГИЙ»

Курсовая работа

Тема: Разработка интернет-магазина одежды

Разработал _____ Зайцева В.В

Руководитель _____ Денисюк А.В

Ангарск, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	8
1.1 Общая характеристика предметной области.....	8
1.2 Анализ существующих решений.....	9
1.2.1 Популярные интернет-магазины	9
1.3 Теоретические основы разработки интернет-магазина одежды	12
1.3.1 Базовые технологии создания веб-сайтов	12
1.3.2 Клиент-серверная архитектура веб-приложения	13
1.4 Выбор технологического стека.....	14
1.4.1 Использование фреймворка Express в серверной части	14
1.4.2 Шаблонизация EJS и организация фронтенда	15
1.4.3 Использование базы данных MySQL	15
1.4.4 Загрузка файлов с использованием Multer	16
1.4.5 Хеширование паролей с использованием bcryptjs	16
1.4.6 Работа с cookie с использованием cookie-parser	17
1.4.7 Авторизация с использованием JSON Web Token (JWT)	17
2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ ИНТЕРНЕТ-МАГАЗИНА ОДЕЖДЫ.....	19
2.1. Назначение и функциональные возможности разработанного приложения	19
2.2. Обоснование применения выбранного технологического стека	20
2.2.1. Серверная платформа и фреймворк	20
2.2.2. Шаблонизатор представлений	21
2.2.3. Система управления базами данных	21
2.2.4. Библиотеки безопасности и вспомогательные зависимости	22
2.3. Архитектура и структура проекта	23
2.3.1. Общая архитектурная модель	23
2.3.2. Точка входа в приложение	23
2.3.3. Структура каталогов проекта	24
2.3.4. Роль маршрутов в архитектуре	25
2.3.5. Контроллеры как центр бизнес-логики.....	26
2.3.6. Middleware как механизм сквозной логики	26

						КР-09.02.07-3-44-ПЗ						
Изм.	Кол.уч	Лист	№ док.	Подпись	Дата							
Разраб.		Зайцева В.В				Разработка интернет-магазина одежды	<u>Лист</u>	Лист	Листов			
Пров.		Денисюк А.В						3	48			
Н.контр												
Утв.							ГАПОУ ИО АТСТ					

2.4. Модель данных и организация базы данных	27
2.4.1 Структура базы данных	27
2.4.2 Связи между таблицами	28
2.5. Реализация регистрации, аутентификации и авторизации	29
2.5.1. Сценарий регистрации пользователя	29
2.5.2. Хеширование пароля и его назначение.....	29
2.5.3. Сценарий входа в систему.....	30
2.5.4. Использование JWT в проекте.....	31
2.5.5. Разграничение прав доступа	32
2.6. Реализация каталога товаров и карточки товара	32
2.6.1. Каталог как центральная часть пользовательского интерфейса.....	32
2.6.2. Фильтрация и поиск в каталоге	33
2.6.3. Сортировка и пагинация.....	33
2.6.4. Карточки товаров	34
2.6.5. Подробная страница товара	34
2.7. Реализация корзины	35
2.7.1. Общая логика работы корзины	35
2.7.2. Добавление товара в корзину.....	35
2.7.4. Отображение корзины	36
2.8. Реализация оформления заказа	37
2.8.1. Назначение подсистемы заказов.....	37
2.8.2. Проверка данных перед созданием заказа.....	37
2.8.3. Использование транзакции.....	37
2.8.4. Формирование записи заказа	38
2.8.5. Просмотр истории заказов	38
2.9. Реализация административной панели	39
2.9.1. Общая роль административного раздела.....	39
2.9.2. Панель управления товарами.....	39
2.9.3. Добавление нового товара.....	40
2.9.4. Алгоритм разбора размерной строки	40
2.9.5. Редактирование товара	41
2.9.6. Удаление товара	41
2.9.7. Просмотр заказов и пользователей.....	41
2.10. Реализация профиля пользователя	42
2.10.1. Назначение личного кабинета.....	42
2.10.2. Редактирование личных данных.....	42

						КР-09.02.07-3-44-ПЗ					
Изм.	Кол.уч	Лист	№ док.	Подпись	Дата						
Разраб.		Зайцева В.В				Разработка интернет-магазина одежды	<u>Лит.</u>	Лист	Листов		
Пров.		Денисюк А.В						4	48		
Н.контр											
Утв.							ГАПОУ ИО АТСТ				

2.10.3. Смена пароля	42
2.11. Реализация пользовательского интерфейса	43
2.11.1. Общий подход к интерфейсу	43
2.11.2. Шапка и навигация.....	43
2.11.3. Каталог и карточки товаров	44
2.11.4. Страницы форм.....	44
2.11.5. Адаптивность интерфейса.....	44
2.12. Обработка ошибок и валидация данных.....	44
2.12.1. Валидация входных данных.....	44
2.12.2. Централизованная обработка ошибок.....	45
2.13. Результаты работы программы.....	45
2.13.1. Практический результат реализации.....	45
2.13.2. Значение реализованных механизмов.....	46
ЗАКЛЮЧЕНИЕ	47
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	49
ПРИЛОЖЕНИЕ А	
ПРИЛОЖЕНИЕ Б.....	
ПРИЛОЖЕНИЕ В	
ПРИЛОЖЕНИЕ Г	
ПРИЛОЖЕНИЕ Д	

						КР-09.02.07-3-44-ПЗ					
Изм.	Кол.уч	Лист	№ док.	Подпись	Дата						
Разраб.		Зайцева В.В				Разработка интернет-магазина одежды	<u>Лист</u>		Лист	Листов	
Пров.		Денисюк А.В						5	48		
Н.контр											
Утв.							ГАПОУ ИО АТСТ				

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий онлайн-торговля прочно вошла в повседневную жизнь современного человека. Одним из наиболее динамично развивающихся сегментов электронной коммерции является продажа одежды через интернет. Потребители всё чаще отдают предпочтение онлайн-покупкам из-за их удобства, экономии времени и широкого выбора. Статистика показывает, что рынок онлайн-ритейла одежды в России продолжает расти, что связано с улучшением качества интернет-услуг, развитием служб доставки и растущим доверием пользователей к онлайн-платежам.

Тема исследования является актуальной ввиду формирующегося рыночного спроса на цифровые платформы и потенциала современных веб-технологий для реализации таких проектов.

Целью данной курсовой работы является создание полноценного интернет-магазина одежды, включающего основные функциональные модули: каталог продукции с возможностью поиска и фильтрации, систему управления корзиной, личный кабинет покупателя и панель администрирования для управления ассортиментом.

Задачи проекта:

1. Изучить современные подходы к разработке интернет-магазинов и проанализировать успешные кейсы в сфере онлайн-продаж одежды.
2. Обосновать выбор технологического стека для реализации проекта.
3. Спроектировать архитектуру приложения с разделением на клиентскую и серверную части.
4. Разработать структуру базы данных для хранения информации о товарах, пользователях и заказах.
5. Реализовать серверное приложение.
6. Создать адаптивный пользовательский интерфейс.

						КР-09.02.07-3-44-26ПЗ	Лист
							6
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Практическая значимость заключается в том, что созданный интернет-магазин может быть использован в качестве учебного проекта для портфолио, а также впоследствии доработан и адаптирован для дальнейшего применения.

						КР-09.02.07-3-44-26ПЗ	Лист
							7
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Общая характеристика предметной области

Предметная область интернет-торговли одеждой представляет собой динамично развивающийся сегмент электронной коммерции, охватывающий онлайн-продажу текстильных товаров, включая повседневную и специализированную одежду, обувь, а также сопутствующие аксессуары. В современных условиях данная сфера приобретает всё большее значение в экономике розничной торговли, трансформируя традиционные модели потребления и создавая новые возможности как для бизнеса, так и для конечных покупателей. Рынок онлайн-продаж одежды характеризуется высокой степенью конкуренции, где наряду с крупными маркетплейсами, такими как Wildberries, Ozon и Ламода, успешно функционируют небольшие нишевые магазины, ориентированные на узкие целевые аудитории и специфические потребительские предпочтения. Сезонность является одной из ключевых особенностей предметной области: ассортиментная политика, маркетинговые кампании и логистические процессы напрямую зависят от времени года, что требует гибкого подхода к управлению запасами и прогнозированию спроса. Особое значение в данной сфере приобретает визуальная составляющая представления товаров — качество фотографий и возможность просмотра изделия с разных ракурсов, демонстрация на модели становятся решающими факторами при принятии покупателем решения о совершении покупки, поскольку онлайн-формат лишает возможности физического контакта с товаром. Не менее важной характеристикой является многообразие параметров описания продукции: одежда требует детальной систематизации по размерным рядам, цветовой гамме, составу материалов, сезонной принадлежности, стилевым направлениям и брендовой идентичности. Все эти особенности формируют специфические требования к архитектуре программных решений в данной предметной области: необходимость реализации гибких систем фильтрации и поиска, поддержки

						КР-09.02.07-3-44-26ПЗ	Лист
							8
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

вариаций товара (размеры, цвета), интеграции с системами управления складскими запасами, а также обеспечения высокой производительности при работе с медиаконтентом. Успешное функционирование интернет-магазина одежды невозможно без комплексного подхода, объединяющего удобный пользовательский интерфейс, надёжную серверную инфраструктуру и продуманную структуру хранения данных, что обуславливает выбор современных технологий разработки, способных обеспечить масштабируемость, безопасность и высокую производительность системы в условиях постоянно растущих требований рынка электронной коммерции.

1.2 Анализ существующих решений

1.2.1 Популярные интернет-магазины

Современный рынок онлайн-торговли одеждой представлен множеством платформ различного масштаба и специализации, каждая из которых реализует собственные подходы к организации пользовательского опыта и бизнес-процессов. Среди лидеров российского сегмента электронной коммерции выделяется маркетплейс Wildberries, занявший доминирующие позиции благодаря масштабному ассортименту, охватывающему миллионы наименований одежды от масс-маркета до премиальных брендов, а также развитой логистической инфраструктуре с пунктами выдачи заказов в тысячах населённых пунктов и удобной системой возвратов, позволяющей покупателям примерить одежду перед окончательным приобретением. Однако данная платформа характеризуется жёсткой конкуренцией внутри площадки, где небольшому продавцу крайне сложно выделиться среди тысяч аналогичных предложений.

С технической точки зрения веб-сервис Wildberries реализован с использованием современных подходов. Интерфейс построен на принципах одностраничного приложения (SPA) с динамической подгрузкой контента при прокрутке каталога. Особое внимание уделено системе фильтрации — реализованы множественные фасетные фильтры, позволяющие

						КР-09.02.07-3-44-26ПЗ	Лист
							9
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

комбинировать параметры поиска (цена, размер, цвет, бренд, рейтинги) без полной перезагрузки страницы. Для работы с медиаконтентом используется адаптивная загрузка изображений в различных разрешениях в зависимости от устройства пользователя.

Компания Ламода представляет собой профильный онлайн-ритейлер, специализирующийся исключительно на одежде, обуви и аксессуарах, что позволило ей сформировать репутацию эксперта в данной категории товаров. Платформа отличается высоким качеством визуального представления продукции: профессиональная фотосъемка на моделях различных типажей, возможность просмотра товара в нескольких ракурсах, а также услуга «примерочной доставки», при которой курьер привозит покупателю сразу несколько размеров и моделей для выбора на дому. Тем не менее, ассортимент Ламоды, несмотря на его разнообразие, уступает по объёму маркетплейсам, а политика ценообразования ориентирована преимущественно на средний и премиальный сегменты, что ограничивает доступность для части потребителей.

Веб-интерфейс Ламоды отличается продуманным пользовательским опытом и высокой степенью интерактивности. Платформа использует технологию бесконечной прокрутки в каталогах, что создаёт эффект непрерывного просмотра и повышает вовлечённость пользователей. Для ускорения навигации реализована предварительная загрузка (prefetching) данных о товарах, находящихся в зоне видимости пользователя. Система поиска включает функцию автодополнения с учётом популярных запросов и исправления опечаток. Особое внимание уделено визуализации размерной сетки — для каждого товара представлена интерактивная таблица размеров с возможностью конвертации между международными стандартами (RU, EU, US, UK). К тому же, внедрена система персонализированных рекомендаций на основе истории просмотров и покупок, а также динамическое отображение остатков товара на складе в реальном времени.

						КР-09.02.07-3-44-26ПЗ	Лист
							10
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Многопрофильная платформа Ozon изначально развивалась как книжный онлайн-магазин, но впоследствии трансформировалась в универсальный маркетплейс, включая категорию одежды в свой ассортимент. Преимуществами платформы являются развитая собственная логистика с возможностью доставки в отдалённые регионы и программа лояльности Ozon Premium. Вместе с тем, одежда не является профильной категорией для Ozon, что отражается на глубине ассортимента, качестве карточек товаров и уровне сервиса в сравнении со специализированными платформами — зачастую отсутствует детальная размерная сетка, ограничен выбор цветовых вариаций, а возвраты обрабатываются дольше, чем у узкоспециализированных конкурентов.

Техническая реализация веб-сервиса Ozon основана на микросервисной архитектуре, что обеспечивает высокую отказоустойчивость и возможность независимого масштабирования отдельных модулей платформы. Фронтенд построен с использованием современных фреймворков, обеспечивающих быстрый отклик интерфейса и плавную анимацию переходов между разделами. Система поиска Ozon интегрирована с машинным обучением для упорядочивания результатов на основе поведенческих факторов и сезонных трендов. Интерфейс адаптирован для работы на различных устройствах. Для повышения лояльности пользователей реализована система уведомлений о снижении цены на интересующие товары, поступлении новых коллекций и статусе заказов в режиме реального времени.

Помимо крупных платформ, рынок включает множество небольших нишевых интернет-магазинов, ориентированных на конкретные стилевые направления (винтажная одежда, спортивный стритстайл, эко-текстиль), целевые группы (одежда для беременных, плюс-сайз, униформа) или локальные бренды. Такие проекты компенсируют отсутствие масштаба за счёт персонализированного подхода к клиентам, уникального ассортимента

						КР-09.02.07-3-44-26ПЗ	Лист
							11
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

и гибкой маркетинговой политики. Анализ существующих решений показывает, что крупные платформы эффективно обслуживают массовый спрос, но оставляют незаполненной нишу специализированных магазинов с узкой целевой аудиторией, где важны не масштаб и разнообразие, а экспертность, уникальность предложений и качество клиентского сервиса. Именно эта ниша представляет наибольший интерес для реализации учебного проекта интернет-магазина одежды, поскольку позволяет продемонстрировать комплексную реализацию ключевых функциональных модулей электронной коммерции без необходимости воспроизводить инфраструктуру масштаба маркетплейсов, сохраняя при этом практическую ценность разработанного решения.

1.3 Теоретические основы разработки интернет-магазина одежды

1.3.1 Базовые технологии создания веб-сайтов

Создание интернет-магазина одежды начинается с использования базовых технологий веб-разработки, которые обеспечивают отображение информации и взаимодействие пользователя с системой. В рассматриваемом проекте используются HTML, CSS и JavaScript как основа клиентской части приложения.

HTML (HyperText Markup Language) применяется для структурирования веб-страниц. С его помощью формируются основные элементы интерфейса: каталог товаров, карточки товаров, формы регистрации и авторизации, корзина покупок и страницы оформления заказа. Каждая страница представляет собой логически организованную структуру, где выделены области навигации, основного контента и вспомогательных элементов.

CSS (Cascading Style Sheets) используется для оформления пользовательского интерфейса. В интернет-магазине одежды CSS отвечает за внешний вид страниц: расположение элементов, цветовую схему, типографику, адаптацию интерфейса под различные устройства. За счёт

						KP-09.02.07-3-44-26ПЗ	Лист
							12
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

применения каскадных стилей достигается единый визуальный стиль всего приложения.

JavaScript применяется для добавления интерактивности. В рамках проекта он используется для обработки пользовательских действий на стороне клиента, таких как отправка форм, динамическое обновление элементов интерфейса и взаимодействие с сервером через HTTP-запросы. Это позволяет улучшить пользовательский опыт без необходимости полной перезагрузки страницы.

Таким образом, HTML, CSS и JavaScript формируют основу клиентской части интернет-магазина, обеспечивая корректное отображение и базовую интерактивность.

1.3.2 Клиент-серверная архитектура веб-приложения

Рассматриваемый интернет-магазин реализован на основе клиент-серверной архитектуры, которая предполагает разделение системы на две логические части: клиент и сервер.

Клиентом выступает веб-браузер пользователя, через который осуществляется доступ к приложению. Пользователь отправляет HTTP-запросы, например, при открытии страницы каталога, добавлении товара в корзину или авторизации. Сервер принимает эти запросы, обрабатывает их и возвращает сформированный ответ.

Серверная часть реализована на платформе Node.js с использованием фреймворка Express. Она отвечает за обработку маршрутов, выполнение бизнес-логики и взаимодействие с базой данных. Например, при добавлении товара в корзину сервер получает соответствующий запрос, проверяет авторизацию пользователя, обновляет данные в базе и возвращает результат операции.

Обмен данными между клиентом и сервером осуществляется по протоколу HTTP. Сервер формирует HTML-страницы с использованием шаблонов и отправляет их клиенту, где они отображаются в браузере. Такой

						КР-09.02.07-3-44-26ПЗ	Лист
							13
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

подход характерен для серверного рендеринга, используемого в данном проекте.

Таким образом, клиент-серверная архитектура обеспечивает разделение ответственности и позволяет эффективно организовать работу интернет-магазина.

1.4 Выбор технологического стека

1.4.1 Использование фреймворка Express в серверной части

Фреймворк Express является ключевым инструментом для разработки серверной части интернет-магазина. Он предоставляет удобные средства для обработки HTTP-запросов, организации маршрутизации и внедрения промежуточного программного обеспечения (middleware).

В рамках проекта Express используется для определения маршрутов, соответствующих различным разделам интернет-магазина: каталог товаров, корзина, заказы, авторизация пользователей и административная панель. Каждый маршрут обрабатывает определённый тип запроса и выполняет соответствующие действия.

Одной из важных особенностей Express является поддержка middleware. В проекте используются промежуточные функции для обработки куков, проверки авторизации пользователя, обработки ошибок и работы с данными запроса. Например, middleware может извлекать информацию о пользователе из токена и добавлять её в объект запроса для последующего использования.

Express также позволяет интегрировать сторонние библиотеки, такие как средства работы с файлами, шифрованием и базами данных, что делает его удобным инструментом для разработки комплексных веб-приложений.

1.4.2 Шаблонизация EJS и организация фронтенда

Для формирования пользовательского интерфейса в проекте используется шаблонизатор EJS (Embedded JavaScript). Он позволяет генерировать HTML-страницы на сервере с использованием встроенного JavaScript-кода.

Основное преимущество EJS заключается в возможности динамического формирования контента. Например, список товаров в каталоге формируется на основе данных, полученных из базы данных. Шаблон содержит разметку HTML и специальные конструкции для вставки данных, таких как названия товаров, цены и изображения.

Использование EJS позволяет объединить серверную и клиентскую части, обеспечивая передачу данных напрямую в шаблон без необходимости использования отдельного API. Это упрощает архитектуру приложения и делает код более компактным.

Фронтенд проекта представлен статическими ресурсами (CSS, JavaScript, изображения) и шаблонами EJS. Такой подход соответствует классической модели серверного рендеринга, где основная логика генерации интерфейса сосредоточена на сервере.

1.4.3 Использование базы данных MySQL

Для хранения данных в интернет-магазине используется реляционная база данных MySQL. Она обеспечивает структурированное хранение информации и возможность выполнения сложных запросов.

В рамках проекта база данных содержит таблицы, связанные с пользователями, товарами, заказами и корзиной. Каждая таблица имеет определённую структуру и связи с другими таблицами. Например, таблица пользователей содержит информацию о зарегистрированных клиентах, а таблица заказов — данные о совершённых покупках.

Взаимодействие с базой данных осуществляется через библиотеку mysql2. Сервер выполняет SQL-запросы для получения, добавления,

изменения и удаления данных. Например, при оформлении заказа данные о выбранных товарах сохраняются в базе, а при входе пользователя выполняется проверка его учетных данных.

Использование MySQL позволяет обеспечить целостность данных и эффективное управление информацией в системе.

1.4.4 Загрузка файлов с использованием Multer

В интернет-магазине одежды реализована возможность загрузки файлов, в частности изображений товаров. Для этого используется библиотека Multer, предназначенная для обработки multipart/form-data запросов.

Multer позволяет принимать файлы, отправленные через формы, и сохранять их на сервере. В проекте изображения товаров загружаются через административную панель и сохраняются в отдельной директории. После загрузки файл становится доступным для отображения на страницах каталога.

Настройка Multer включает определение места хранения файлов, их именования и фильтрации. Это позволяет контролировать процесс загрузки и обеспечивать безопасность системы.

Таким образом, использование Multer обеспечивает функциональность управления медиафайлами в интернет-магазине.

1.4.5 Хеширование паролей с использованием bcryptjs

Для обеспечения безопасности пользовательских данных в проекте применяется библиотека bcryptjs. Она используется для хеширования паролей перед их сохранением в базе данных.

При регистрации пользователя введенный пароль не сохраняется в открытом виде. Вместо этого он преобразуется в хеш с использованием криптографического алгоритма. При последующей авторизации введенный пароль сравнивается с сохранённым хешем.

Использование `bcryptjs` позволяет защитить учетные записи пользователей от компрометации в случае утечки данных. Даже при получении доступа к базе данных злоумышленник не сможет восстановить исходные пароли.

Перед сохранением в базу данных пароль преобразуется в хеш. Число 10 задаёт сложность хеширования. В результате в базе хранится не исходный пароль, а его защищённое представление.

1.4.6 Работа с cookie с использованием cookie-parser

В проекте используется библиотека `cookie-parser` для обработки cookie — небольших фрагментов данных, хранящихся в браузере пользователя.

Cookie применяются для хранения информации о сессии пользователя, в частности токена авторизации. При каждом запросе браузер отправляет cookie на сервер, где они анализируются и используются для определения состояния пользователя.

Библиотека `cookie-parser` упрощает работу с cookie, предоставляя удобный интерфейс для их чтения и записи. Это позволяет реализовать механизм авторизации и поддерживать состояние пользователя между запросами.

1.4.7 Авторизация с использованием JSON Web Token (JWT)

Для реализации механизма авторизации в интернет-магазине используется технология JSON Web Token (JWT). Она позволяет безопасно передавать информацию о пользователе между клиентом и сервером.

JWT представляет собой токен, содержащий закодированную информацию о пользователе. После успешной авторизации сервер генерирует токен и отправляет его клиенту, где он сохраняется в cookie. При последующих запросах токен передаётся на сервер и проверяется на подлинность.

Внутри токена может содержаться идентификатор пользователя и другая служебная информация. Проверка подписи токена гарантирует, что он не был изменён.

Использование JWT позволяет реализовать безсессионную авторизацию, при которой сервер не хранит информацию о сессии пользователя. Это упрощает масштабирование приложения и повышает его производительность.

						КР-09.02.07-3-44-26ПЗ	Лист
							18
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ ИНТЕРНЕТ-МАГАЗИНА ОДЕЖДЫ

2.1. Назначение и функциональные возможности разработанного приложения

Практическая часть курсовой работы посвящена реализации веб-приложения интернет-магазина одежды. Назначение системы состоит в автоматизации базовых процессов электронной торговли: публикации каталога товаров, выбора товара пользователем, добавления в корзину, оформления заказа, управления личными данными клиента, а также администрирования ассортимента и просмотра заказов. В файле `package.json` проект прямо описан как «Интернет-магазин одежды», а серверная конфигурация подключает маршруты магазина, авторизации, корзины, заказов, администрирования и профиля, что подтверждает полнофункциональный характер системы.

С точки зрения пользовательских сценариев приложение поддерживает: просмотр каталога `/catalog`, открытие детальной страницы товара `/products/:id`, регистрацию и вход через `/auth/register` и `/auth/login`, работу с корзиной `/cart`, оформление заказа `/orders/create`, просмотр списка собственных заказов `/orders`, редактирование профиля `/profile`, а для администратора — просмотр панели управления `/admin`, создание, изменение и удаление товаров, а также просмотр заказов и пользователей. Эти возможности распределены по отдельным маршрутам и контроллерам, что видно и подключений в `server.js` и определений соответствующих роутов.

Визуально приложение ориентировано на традиционный сценарий интернет-магазина: в шапке сайта размещены переходы к каталогу, корзине, профилю и админ-панели в зависимости от роли пользователя; каталог выводит карточки товаров с изображением, названием, ценой и ссылкой на детальную страницу; страница товара содержит выбор размера и количества; корзина показывает состав заказа и форму ввода адреса доставки; личный

						КР-09.02.07-3-44-26ПЗ	Лист
							19
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

кабинет включает редактирование контактных данных и смену пароля. Это следует из шаблонов header.ejs, catalog.ejs, product.ejs, cart/index.ejs и profile/index.ejs.

2.2. Обоснование применения выбранного технологического стека

2.2.1. Серверная платформа и фреймворк

Для реализации серверной части приложения выбрана платформа Node.js. Она позволяет выполнять JavaScript-код на стороне сервера и эффективно обрабатывать большое количество запросов благодаря асинхронной модели ввода-вывода. Использование Node.js особенно удобно в учебных проектах, так как позволяет применять один язык программирования как на клиентской, так и на серверной стороне.

В качестве веб-фреймворка используется Express. Он обеспечивает гибкую маршрутизацию, поддержку middleware, удобную обработку HTTP-запросов и возможность модульной организации серверного кода. В рамках данного проекта Express выступает центральным связующим звеном между маршрутизацией, контроллерами, middleware и представлениями.

Применение Express обосновано следующими причинами:

- простота настройки и запуска;
- возможность разделения кода по модулям;
- наличие большого количества готовых библиотек;
- широкое распространение в учебных и прикладных проектах;
- удобная интеграция с шаблонизаторами и базами данных.

В данном проекте Express используется как основа всей серверной инфраструктуры: в файле server.js создаётся экземпляр приложения, подключаются middleware, шаблонизатор, статические каталоги и модули маршрутов.

2.2.2. Шаблонизатор представлений

Для формирования пользовательского интерфейса выбран шаблонизатор EJS. Он позволяет генерировать HTML-страницы на стороне сервера с использованием переменных, условий и циклов. Благодаря этому данные, полученные из базы данных, могут быть напрямую встроены в шаблон страницы.

Использование EJS в данном проекте обусловлено тем, что система строится как серверное веб-приложение, а не как отдельный клиентский интерфейс на React или Vue. Такой подход упрощает архитектуру и делает проект более понятным для учебной разработки.

Шаблонизатор применяется для:

- вывода каталога товаров;
- отображения карточки конкретного товара;
- формирования страниц входа и регистрации;
- показа корзины и заказов;
- вывода административных страниц;
- построения шапки и подвала сайта.

Таким образом, EJS обеспечивает тесную интеграцию между серверной логикой и представлением данных.

2.2.3. Система управления базами данных

В качестве СУБД используется MySQL. Это реляционная система управления базами данных, которая хорошо подходит для хранения структурированных данных интернет-магазина: пользователей, товаров, корзины, заказов и размерных остатков.

Подключение к MySQL реализовано через библиотеку `mysql2/promise`, что позволяет использовать асинхронную работу с базой данных в стиле `async/await`. Для повышения стабильности работы используется пул соединений.

Выбор MySQL обоснован следующими факторами:

- поддержка реляционной модели данных;
- возможность организации связей между таблицами;
- высокая производительность при типовых запросах;
- широкое распространение;
- простота развёртывания в учебных проектах.

В рассматриваемом проекте база данных играет ключевую роль, так как практически все функции приложения зависят от хранения и извлечения структурированных данных.

2.2.4. Библиотеки безопасности и вспомогательные зависимости

Помимо основных технологий, в проекте используются дополнительные библиотеки, каждая из которых выполняет отдельную прикладную задачу.

1. Библиотека `bcryptjs` применяется для хеширования паролей. Это позволяет не хранить пароли в открытом виде и тем самым повышает безопасность системы.
2. Библиотека `jsonwebtoken` используется для реализации авторизации через JWT. После успешного входа пользователь получает токен, который сохраняется в `cookie` и передаётся серверу при последующих запросах.
3. Библиотека `cookie-parser` обеспечивает обработку `cookie` на стороне Express-сервера.
4. Библиотека `express-validator` применяется для валидации входных данных форм. Она используется в сценариях регистрации, входа, изменения профиля, оформления заказа и административного редактирования товаров.
5. Библиотека `multer` отвечает за загрузку изображений товаров. Она обрабатывает `multipart`-запросы и сохраняет файлы в каталог `public/uploads`.

6. Библиотека `method-override` позволяет имитировать HTTP-методы `PUT` и `DELETE` из HTML-форм, которые по стандарту поддерживают только `GET` и `POST`.
7. Библиотека `dotenv` используется для загрузки переменных окружения из файла `.env`, что позволяет хранить параметры подключения к базе данных и другие конфигурационные значения вне исходного кода.

2.3. Архитектура и структура проекта

2.3.1. Общая архитектурная модель

Архитектурно приложение реализовано по модульному принципу с выраженным разделением на маршруты, контроллеры, шаблоны представлений, вспомогательные утилиты, `middleware` и слой доступа к данным. На верхнем уровне репозитория присутствуют каталоги `config`, `controllers`, `middlewares`, `public`, `routes`, `utils`, `views`, а также файлы `server.js`, `database.sql`, `package.json` и инструкции по запуску в `README`. Такая структура характерна для классического серверного MVC-подхода и облегчает сопровождение кода.

2.3.2. Точка входа в приложение

Центральной точкой входа является `server.js`. Здесь выполняется инициализация Express-приложения, регистрация `middleware` и маршрутов. После базовой настройки сервер подключает `attachUser`, который извлекает данные текущего пользователя из JWT и делает их доступными как в `req.user`, так и в `res.locals.currentUser`. Благодаря этому шаблоны EJS могут динамически менять навигацию и набор кнопок без повторной ручной передачи пользователя в каждый `res.render`.

```
const express = require('express');  
  
const app = express();  
  
app.set('view engine', 'ejs');  
  
app.use(express.urlencoded({ extended: true }));  
  
app.use(express.static('public'));
```

Данный фрагмент показывает базовую настройку Express-приложения. Сначала создаётся объект приложения, затем подключается шаблонизатор EJS, обработка данных из HTML-форм и каталог public для статических файлов.

2.3.3. Структура каталогов проекта

Структура проекта организована по функциональному принципу.

Каталог config содержит инфраструктурные настройки. В нём размещаются файлы конфигурации базы данных и загрузки файлов.

Каталог controllers содержит контроллеры, в которых реализована основная бизнес-логика приложения:

- shop.controller.js
- auth.controller.js
- cart.controller.js
- order.controller.js
- admin.controller.js
- profile.controller.js

Каталог routes содержит модули маршрутов, сопоставляющие URL с действиями контроллеров:

- shop.routes.js
- auth.routes.js
- cart.routes.js
- order.routes.js
- admin.routes.js
- profile.routes.js

Каталог middlewares содержит промежуточные обработчики:

- аутентификация и авторизация
- валидация
- обработка ошибок

Каталог `utils` содержит служебные функции и небольшие переиспользуемые компоненты:

- работа с JWT и cookie
- класс прикладной ошибки
- обёртка для асинхронных контроллеров

Каталог `views` содержит EJS-шаблоны интерфейса, сгруппированные по функциональным разделам:

- `auth`
- `shop`
- `cart`
- `admin`
- `profile`
- `partials`

Каталог `public` содержит статические ресурсы: CSS-файлы, изображения и каталог `uploads` для загруженных файлов товаров.

2.3.4. Роль маршрутов в архитектуре

Маршруты являются первым прикладным уровнем после получения HTTP-запроса сервером. Они выполняют функцию распределения запросов по соответствующим модулям приложения. Каждый файл маршрутов отвечает за отдельную предметную область.

Маршруты магазина отвечают за главную страницу, каталог и карточку товара. Маршруты авторизации обрабатывают регистрацию, вход и выход. Маршруты корзины и заказов реализуют сценарии покупки. Маршруты профиля управляют личными данными пользователя. Маршруты администратора предоставляют доступ к служебным операциям управления магазином.

						КР-09.02.07-3-44-26ПЗ	Лист
							25
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

2.3.5. Контроллеры как центр бизнес-логики

Бизнес-логика сосредоточена в контроллерах. `shop.controller.js` обслуживает витрину магазина: редиректит главную страницу на каталог, формирует SQL-запросы с фильтрацией и сортировкой, извлекает детальную информацию о товаре и его размерах. `auth.controller.js` отвечает за регистрацию, вход и выход пользователя. `cart.controller.js` реализует работу с корзиной: просмотр, добавление, изменение количества и удаление. `order.controller.js` оформляет заказ в транзакции и выводит историю заказов. `profile.controller.js` изменяет личные данные и пароль. `admin.controller.js` управляет ассортиментом, а также получает списки заказов и пользователей для административных экранов.

Слой представлений организован в каталоге `views`, где выделены подкаталоги `admin`, `auth`, `cart`, `partials`, `profile` и `shop`. Это означает, что визуальная часть также разбита по подсистемам. Общие элементы интерфейса, например шапка и подвал сайта, вынесены в `partials`, а страницы каталога, товара, корзины, заказа, профиля и админ-панели описаны отдельными EJS-файлами. Такое решение делает интерфейс повторно используемым и уменьшает дублирование HTML-кода.

2.3.6. Middleware как механизм сквозной логики

Middleware в данном проекте выполняют кросс-секционные задачи, необходимые во многих местах приложения. Они позволяют не дублировать однотипные проверки в каждом контроллере.

Основные функции middleware:

- определение текущего пользователя по JWT-токену
- запрет доступа гостю к закрытым разделам
- запрет доступа авторизованному пользователю к гостевым страницам
- ограничение доступа к административным разделам
- проверка корректности входных данных

						КР-09.02.07-3-44-26ПЗ	Лист
							26
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

– единая обработка ошибок

Благодаря этому контроллеры остаются сосредоточенными на бизнес-логике, а общие функции вынесены в отдельный слой.

2.4. Модель данных и организация базы данных

2.4.1 Структура базы данных

База данных проекта описана в database.sql. В системе используются таблицы users, products, product_sizes, cart_items, orders и order_items. Такое проектирование отражает специфику интернет-магазина одежды: один товар может выпускаться в нескольких размерах, а значит остатки следует хранить не на уровне товара, а на уровне связки «товар–размер». Именно поэтому в схеме присутствует отдельная таблица product_sizes, связанная с products отношением один-ко-многим.

Таблица users хранит имя, email, хеш пароля, роль, телефон, город, адрес, дату создания и дату обновления. Роль задается перечислением ENUM('user', 'admin'), что позволяет использовать одну и ту же таблицу для обычных клиентов и администраторов. Наличие дополнительных полей phone, city и address прямо связано с профилем пользователя и упрощает хранение контактной информации без выделения отдельной таблицы профиля.

Таблица products хранит основные атрибуты товара: название, описание, категорию, тип по полу (male, female, unisex), цену, путь к изображению, признак активности и временные метки. Поле is_active используется в каталоге: публичная витрина выводит только активные товары, а администратор при редактировании может активировать или деактивировать позицию. Это видно как в SQL-схеме, так и в контроллерах каталога и админ-панели.

Корзина реализована через таблицу cart_items, где хранятся пользователь, товар, выбранный размер и количество. Уникальный ключ unique_cart_item (user_id, product_size_id) гарантирует, что у одного

						КР-09.02.07-3-44-26ПЗ	Лист
							27
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

пользователя не будет нескольких отдельных строк корзины для одного и того же размерного варианта: вместо дублирования строк контроллер увеличивает количество уже существующей позиции. Это решение упрощает бизнес-логику и делает интерфейс корзины предсказуемым.

Оформленный заказ сохраняется в таблице `orders`, а его состав — в `order_items`. В `orders` хранятся общий итог, статус, адрес доставки и комментарий, а в `order_items` — товар, размер, цена на момент заказа и количество. Такой подход важен с точки зрения учета: даже если впоследствии карточка товара будет изменена, история заказа сохранит точное название, размер и цену в момент покупки. Это подтверждается структурой таблиц и SQL-вставкой, выполняемой в `order.controller.js`.

2.4.2 Связи между таблицами

Структура базы данных строится на нескольких ключевых отношениях:

- один пользователь может иметь много элементов корзины;
- один пользователь может иметь много заказов;
- один товар может иметь много размеров;
- один заказ содержит много позиций;
- одна размерная позиция товара может встречаться во многих корзинах и заказах.

Это обеспечивает логическую целостность данных и отражает реальные бизнес-процессы интернет-магазина.

Ег-диаграмма базы данных см. Приложение А. Диаграмма отражает реальные таблицы и внешние ключи из `database.sql`.

2.5. Реализация регистрации, аутентификации и авторизации

2.5.1. Сценарий регистрации пользователя

Регистрация пользователя реализована через HTML-форму, где вводятся имя, адрес электронной почты, пароль и подтверждение пароля. После отправки формы браузер направляет POST-запрос на соответствующий маршрут, который обрабатывается контроллером авторизации.

Сначала сервер проверяет корректность введенных данных. На этом этапе контролируется длина имени, формат email и минимальная длина пароля. Также проверяется наличие поля подтверждения пароля. После этого контроллер дополнительно убеждается, что пароль и подтверждение совпадают.

Следующий этап — проверка уникальности пользователя. Для этого выполняется запрос к таблице users, который ищет запись с указанным email. Если такой пользователь уже существует, регистрация отклоняется, а пользователю возвращается сообщение об ошибке.

Если адрес уникален, пароль хешируется с помощью библиотеки bcryptjs. Это означает, что исходное значение пароля не сохраняется в базе данных. Вместо него в таблицу попадает криптографический хэш. Такой подход является стандартом безопасного хранения паролей.

После хеширования в таблицу users добавляется новая запись с ролью user. Затем сервер автоматически создаёт JWT-токен и помещает его в cookie. Это позволяет пользователю сразу после регистрации войти в систему без дополнительного шага авторизации.

2.5.2. Хеширование пароля и его назначение

Хеширование пароля является важным элементом безопасности проекта. В отличие от симметричного шифрования, хеширование необратимо: из хэша невозможно восстановить исходный пароль.

						КР-09.02.07-3-44-26ПЗ	Лист
							29
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

При регистрации система вызывает функцию `bcrypt.hash(password, 10)`, где число 10 определяет сложность вычислений. Более высокий параметр делает подбор хэша труднее для злоумышленника, но увеличивает нагрузку на сервер. Для учебного проекта выбранное значение является разумным компромиссом между безопасностью и производительностью.

```
const hashedPassword = await bcrypt.hash(password, 10);
```

При входе в систему сервер не расшифровывает пароль, а сравнивает введённое пользователем значение с хэшем при помощи `bcrypt.compare`. Если сравнение успешно, пользователь считается подлинным.

Таким образом, даже при утечке базы данных злоумышленник не получает пароли пользователей в открытом виде.

2.5.3. Сценарий входа в систему

Вход в систему выполняется через форму авторизации. Пользователь вводит email и пароль, после чего данные отправляются на сервер.

При авторизации введённый пользователем пароль сравнивается с хэшем, который хранится в базе данных. Если сравнение успешно, пользователь получает доступ к системе.

```
const isMatch = await bcrypt.compare(password, user.password);
```

Контроллер выполняет поиск пользователя по email. Если запись отсутствует, доступ отклоняется. Если пользователь найден, система сравнивает введённый пароль с сохранённым в базе хэшем. При совпадении создаётся JWT-токен, который содержит идентификатор пользователя и его роль.

Токен помещается в cookie с флагом `httpOnly`. Это означает, что клиентский JavaScript не может получить прямой доступ к токену, что снижает риск его компрометации через XSS-уязвимости.

После успешного входа пользователь перенаправляется на главную страницу каталога.

UML-диаграмма последовательности сценария входа пользователя см. Приложение Б.

Система авторизации не ограничивается входом. В `auth.middleware.js` имеются три специализированные проверки. `isAuthenticated` перенаправляет неавторизованного пользователя на страницу входа. `isGuest` предотвращает доступ аутентифицированного пользователя к гостевым страницам. `isAdmin` сначала требует авторизации, а затем проверяет роль `admin`; при несоответствии выбрасывается `AppError` с кодом 403. Именно этот механизм охраняет административные маршруты приложения.

2.5.4. Использование JWT в проекте

JWT используется в проекте как средство хранения состояния авторизации без применения серверных сессий. Внутри токена содержится минимально необходимая информация: идентификатор пользователя и его роль.

При каждом следующем запросе сервер извлекает токен из `cookie`, проверяет его подлинность и, если он валиден, получает данные пользователя из базы данных. После этого объект пользователя прикрепляется к `req.user`, а также становится доступным в шаблонах через `res.locals.user`.

Это позволяет:

- отображать в интерфейсе состояние авторизации
- определять доступность закрытых маршрутов
- ограничивать административные функции по роли
- связывать корзину и заказы с текущим пользователем

Использование JWT делает архитектуру приложения проще, так как сервер не хранит отдельное состояние сессии для каждого пользователя.

2.5.5. Разграничение прав доступа

- В проекте реализовано ролевое разграничение доступа. Все пользователи делятся на две категории:
- обычные пользователи;
- администраторы.
- Для контроля доступа используются middleware:
- `isAuthenticated` — допускает только авторизованных пользователей;
- `isGuest` — допускает только гостей;
- `isAdmin` — допускает только администраторов.

Благодаря этому пользователь не может попасть в корзину, профиль или заказы без входа в систему. В то же время административная панель недоступна обычному покупателю, даже если он успешно авторизован.

Такое разграничение соответствует реальной логике интернет-магазина и является обязательным элементом прикладной системы.

2.6. Реализация каталога товаров и карточки товара

2.6.1. Каталог как центральная часть пользовательского интерфейса

Пользовательская часть магазина начинается с домашнего маршрута `/`, который не рендерит отдельную страницу, а сразу перенаправляет пользователя на `/catalog`. Это означает, что каталог выступает главной точкой входа в приложение. Сам каталог формируется в `shop.controller.js` на основе параметров запроса: категории, гендерной принадлежности, диапазона цены, поисковой строки, сортировки и номера страницы.

Диаграмма последовательности для просмотра каталога товаров пользователем предоставлена в приложении В.

2.6.2. Фильтрация и поиск в каталоге

Функциональность каталога не ограничивается простым выводом списка товаров. В проекте реализованы фильтрация и поиск по нескольким параметрам:

- категория
- пол
- минимальная цена
- максимальная цена
- строка поиска
- вариант сортировки

На стороне сервера эти параметры извлекаются из строки запроса и используются для динамического формирования SQL-условий. В результате пользователь может гибко управлять содержимым витрины.

Например, если указана категория, контроллер добавляет условие по полю category. Если введён текст поиска, сервер ищет совпадения в названии и описании товара. Если задан ценовой диапазон, дополнительно фильтруются товары по цене.

Такой подход позволяет обрабатывать множество пользовательских комбинаций без написания отдельного маршрута для каждого случая.

2.6.3. Сортировка и пагинация

Помимо фильтрации, каталог поддерживает сортировку по нескольким критериям:

- сначала новые
- цена по возрастанию
- цена по убыванию
- по названию

Также реализована пагинация. Контроллер предварительно вычисляет общее количество товаров, подходящих под условия фильтрации, после чего

определяет число страниц и извлекает только нужную часть результатов через LIMIT и OFFSET.

Пагинация делает каталог пригодным для работы с большим количеством товаров и снижает нагрузку на страницу.

2.6.4. Карточки товаров

Каждый товар отображается в виде карточки. С точки зрения интерфейса карточка товара играет сразу несколько ролей:

- визуально представляет товар
- позволяет быстро оценить цену
- служит точкой перехода к подробной информации
- формирует общее впечатление о качестве интерфейса магазина

В проекте карточки имеют фиксированный размер и единый стиль. Это достигается за счёт CSS-оформления, где задаются размеры блока, высота изображения, скругления, тени и эффекты при наведении.

Фиксированные размеры карточек особенно важны для интернет-магазина одежды, так как изображения товаров могут иметь разные пропорции. Благодаря единым размерам каталог выглядит аккуратно и не распадается визуально.

2.6.5. Подробная страница товара

При переходе по кнопке «Подробнее» пользователь попадает на страницу конкретного товара. Контроллер получает идентификатор товара из URL, извлекает соответствующую запись из таблицы products, а затем запрашивает все размерные позиции из product_sizes.

На странице товара отображаются:

- название
- категория
- описание
- цена

- изображение
- список доступных размеров
- остатки по размерам
- форма добавления в корзину

Если пользователь не авторизован, сама страница товара остаётся доступной, но действия, связанные с покупкой, становятся недоступными до входа в систему. Такой подход позволяет не скрывать товары от гостей, но защищает бизнес-операции.

2.7. Реализация корзины

2.7.1. Общая логика работы корзины

Корзина представляет собой временное хранилище товаров, которые пользователь планирует приобрести. В данном проекте корзина связана с конкретным пользователем, поэтому для работы с ней требуется авторизация.

Каждая позиция корзины содержит ссылку не просто на товар, а на конкретную размерную запись товара. Это означает, что корзина учитывает размер одежды как отдельный параметр покупки.

Такой подход особенно важен для магазина одежды, так как два одинаковых товара разных размеров нельзя считать взаимозаменяемыми.

Диаграмма последовательности добавления товара в корзину пользователем предоставлена в приложении Г.

2.7.2. Добавление товара в корзину

При добавлении товара в корзину пользователь выбирает размер и количество. После отправки формы контроллер выполняет несколько последовательных проверок:

- существует ли выбранная размерная запись;
- активен ли товар;
- достаточно ли товара на складе;

- есть ли уже такая позиция в корзине пользователя.

Если позиция уже присутствует, количество увеличивается. Если нет, создаётся новая запись в таблице `cart_items`.

Наличие уникального ограничения в таблице корзины гарантирует, что один и тот же размер одного и того же товара не появится у пользователя в корзине в виде нескольких отдельных строк.

```
await db.query(
  'INSERT INTO cart_items (user_id, product_id, quantity) VALUES (?, ?, ?)',
  [userId, productId, quantity]
);
```

Рис. 1 Добавление товара в корзину

При добавлении товара в корзину создаётся запись, связывающая пользователя с выбранным товаром и количеством.

2.7.3. Изменение количества и удаление позиции

Пользователь может изменять количество товара в корзине. При этом контроллер повторно сверяет новое количество с доступным остатком на складе. Если количество превышает запас, обновление не выполняется.

Если пользователь задаёт количество, равное нулю или меньше, запись удаляется из корзины. Это позволяет использовать один и тот же механизм как для редактирования количества, так и для удаления позиции.

2.7.4. Отображение корзины

При открытии страницы корзины контроллер извлекает записи пользователя из таблицы `cart_items` и соединяет их с данными из таблиц `products` и `product_sizes`. Благодаря этому в шаблон передаются уже готовые для отображения данные:

- название товара
- изображение
- цена
- выбранный размер

- количество
- итог по позиции
- общая сумма корзины

Визуально корзина оформлена как список карточек или блоков с возможностью обновления и удаления каждой позиции, а также с формой оформления заказа.

2.8. Реализация оформления заказа

2.8.1. Назначение подсистемы заказов

Подсистема заказов завершает пользовательский сценарий покупки. Её задача — преобразовать содержимое корзины в постоянную запись в базе данных, уменьшить остатки на складе и зафиксировать параметры заказа. Без этой подсистемы проект был бы лишь каталогом с корзиной, но не полноценным интернет-магазином.

2.8.2. Проверка данных перед созданием заказа

Перед созданием заказа пользователь должен указать адрес доставки. Это поле проходит валидацию по длине и обязательности. Дополнительно пользователь может оставить комментарий.

На сервере контроллер сначала получает корзину пользователя и убеждается, что она не пуста. Затем каждая позиция проверяется на соответствие актуальному остатку на складе. Это защищает систему от ситуации, когда товар был добавлен в корзину раньше, а затем остаток изменился.

2.8.3. Использование транзакции

Создание заказа затрагивает сразу несколько таблиц:

- orders
- order_items
- product_sizes
- cart_items

Чтобы все эти изменения выполнялись согласованно, используется транзакция базы данных. Это означает, что либо все операции проходят успешно, либо при ошибке откатываются полностью.

Транзакция особенно важна для следующих сценариев:

- одновременное оформление заказов несколькими пользователями
- ошибка при вставке одной из строк состава заказа
- несоответствие складских остатков

Использование транзакции делает оформление заказа атомарной операцией.

2.8.4. Формирование записи заказа

После успешной проверки корзины сервер вычисляет общую сумму заказа и создаёт запись в таблице orders. Затем для каждой позиции корзины создаётся запись в таблице order_items, где фиксируются:

- идентификатор заказа
- товар
- размер
- название товара
- название размера
- цена на момент покупки
- количество

Параллельно в таблице product_sizes уменьшается остаток соответствующего размера. После этого корзина пользователя очищается.

2.8.5. Просмотр истории заказов

После оформления пользователь может открыть страницу заказов. Контроллер извлекает заказы текущего пользователя, а затем для каждого заказа подгружает список его позиций.

На странице истории заказов отображаются:

						КР-09.02.07-3-44-26ПЗ	Лист
							38
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- номер заказа
- статус
- итоговая сумма
- адрес доставки
- дата создания
- состав заказа

Это позволяет пользователю просматривать историю своих покупок и подтверждает факт успешной работы системы заказов.

2.9. Реализация административной панели

2.9.1. Общая роль административного раздела

Административная панель предназначена для управления содержимым магазина и просмотра служебной информации. Она доступна только пользователю с ролью admin.

Наличие отдельного административного контура является обязательной частью проекта интернет-магазина, так как позволяет отделить клиентские сценарии от служебных операций управления системой.

2.9.2. Панель управления товарами

Основной экран административного раздела выводит список товаров. Здесь администратор может:

- просматривать все товары, включая неактивные
- перейти к созданию нового товара
- редактировать существующий товар
- удалить товар

Отображение всех товаров, а не только активных, важно для реального управления ассортиментом.

2.9.3. Добавление нового товара

Для создания товара используется отдельная форма. В ней администратор указывает:

- название
- описание
- категорию
- пол
- цену
- изображение
- строку размеров и остатков
- признак активности

После отправки формы данные проходят валидацию. Далее, если приложен файл изображения, он сохраняется в каталог загрузок. После этого создаётся запись в таблице products.

Отдельно обрабатывается строка размеров. Она преобразуется в набор объектов вида «размер — количество», после чего для каждого элемента создаётся запись в таблице product_sizes.

Таким образом, создание товара в административной панели включает не только сам товар, но и его размерную структуру.

Диаграмма последовательности добавления нового товара администратором см. Приложение Д.

2.9.4. Алгоритм разбора размерной строки

Размеры в форме администратора задаются в компактном текстовом формате. Для обработки этой строки используется вспомогательная функция, которая:

- разбивает строку по запятым
- для каждого элемента выделяет размер и количество
- убирает лишние пробелы

- преобразует количество в число
- отбрасывает пустые или некорректные элементы

Такое решение упрощает пользовательский интерфейс формы и позволяет администратору быстро вводить размерную сетку.

2.9.5. Редактирование товара

Редактирование товара также выполняется через форму. Перед открытием формы контроллер извлекает текущие данные товара и существующие размеры. Затем размеры собираются в строковый формат, пригодный для редактирования в одном поле формы.

При сохранении изменений контроллер обновляет основные поля товара. Если загружено новое изображение, путь к картинке заменяется. После этого старые записи размеров удаляются, а новые создаются заново на основе обновлённой строки размеров.

2.9.6. Удаление товара

Удаление товара выполняется прямым SQL-запросом к таблице products. Благодаря каскадным ограничениям внешних ключей связанные записи в таблице размеров удаляются автоматически.

Это упрощает реализацию и гарантирует целостность данных.

2.9.7. Просмотр заказов и пользователей

Административная панель предоставляет администратору доступ к списку всех заказов. Для каждого заказа выводятся:

пользователь;

- email
- сумма
- статус
- адрес доставки
- состав заказа
- дата создания

Также реализован экран просмотра пользователей. На нём отображаются зарегистрированные пользователи и количество заказов, оформленных каждым из них.

Эти функции делают административный раздел не только инструментом редактирования товаров, но и средством мониторинга работы магазина.

2.10. Реализация профиля пользователя

2.10.1. Назначение личного кабинета

Личный кабинет предназначен для хранения и изменения персональных данных пользователя. Он дополняет систему авторизации и делает учётную запись не просто средством входа, а полноценным персональным профилем.

2.10.2. Редактирование личных данных

На странице профиля пользователь может изменить:

- имя
- телефон
- город
- адрес

Эти данные сохраняются в таблице users и затем могут быть использованы при оформлении заказов. Например, адрес уже логически связан со сценарием доставки.

2.10.3. Смена пароля

Отдельный блок профиля отвечает за смену пароля. Для этого пользователь должен ввести:

- текущий пароль
- новый пароль
- подтверждение нового пароля

Система проверяет совпадение нового пароля и подтверждения, а затем сверяет текущий пароль с хэшем в базе. Только при успешной проверке выполняется повторное хеширование и обновление поля password_hash.

Таким образом, смена пароля строится на тех же механизмах безопасности, что и первичная регистрация.

2.11. Реализация пользовательского интерфейса

2.11.1. Общий подход к интерфейсу

Интерфейс приложения реализован на основе HTML, EJS и CSS. Он построен по шаблонному принципу: общие элементы шапки и подвала вынесены в частичные представления, а каждая функциональная область имеет собственный шаблон страницы.

Такой подход обеспечивает:

- единообразие оформления
- уменьшение дублирования разметки
- удобство сопровождения интерфейса

2.11.2. Шапка и навигация

Шапка сайта содержит логотип и навигационное меню. Состав меню зависит от состояния пользователя:

- для гостя доступны вход и регистрация
- для авторизованного пользователя доступны корзина, заказы и профиль
- для администратора дополнительно показывается ссылка на админку

Это делает интерфейс адаптивным не только с точки зрения размеров экрана, но и с точки зрения роли пользователя.

2.11.3. Каталог и карточки товаров

Каталог оформлен как сетка карточек товаров. Карточки имеют фиксированный размер, единый стиль, тени и эффекты при наведении. Такой дизайн улучшает восприятие большого количества товаров на странице.

Верхняя часть каталога содержит заголовок и форму фильтрации. Ниже располагается сетка товаров, а под ней — пагинация.

2.11.4. Страницы форм

Формы регистрации, входа, редактирования профиля и администрирования товаров оформлены в виде карточек с полями ввода и кнопками действия. Такой подход делает интерфейс визуально согласованным.

2.11.5. Адаптивность интерфейса

В таблице стилей используются *media queries*, благодаря которым интерфейс перестраивается под меньшие размеры экрана. На мобильных устройствах карточки товаров и некоторые двухколоночные блоки переходят в одноколоночное отображение.

Это повышает удобство использования приложения на разных устройствах.

2.12. Обработка ошибок и валидация данных

2.12.1. Валидация входных данных

Проверка пользовательского ввода выполняется до попадания данных в контроллер. Для этого используются правила *express-validator*. В зависимости от сценария контролируются:

- формат email
- длина имени
- минимальная длина пароля
- корректность числовых значений
- обязательность адреса доставки

- корректность данных товара в административной панели

Такой подход делает работу приложения устойчивее и предотвращает попадание некорректных данных в базу данных.

2.12.2. Централизованная обработка ошибок

Для ошибок в проекте используется специальный класс прикладной ошибки. Если в контроллере возникает исключительная ситуация, например товар не найден или доступ запрещён, создаётся ошибка с нужным HTTP-статусом.

Асинхронные контроллеры обернуты в специальную функцию, которая автоматически передаёт ошибку в общий обработчик Express. Затем пользователь получает либо страницу ошибки, либо страницу 404.

Централизованная обработка ошибок делает приложение более устойчивым и избавляет от необходимости дублировать код try/catch в каждом маршруте.

2.13. Результаты работы программы

2.13.1. Практический результат реализации

В результате разработки была создана рабочая серверная система интернет-магазина одежды. После запуска приложения пользователь может:

- открыть каталог товаров
- отфильтровать и отсортировать товары
- просмотреть карточку товара
- зарегистрироваться и войти в систему
- выбрать размер и добавить товар в корзину
- изменить содержимое корзины
- оформить заказ
- просматривать историю заказов
- редактировать данные профиля

Администратор, в свою очередь, может:

						КР-09.02.07-3-44-26ПЗ	Лист
							45
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- войти в административную панель
- создавать товары
- редактировать товар
- удалять товары
- просматривать список заказов
- просматривать список пользователей

2.13.2. Значение реализованных механизмов

Наиболее значимыми практическими результатами проекта являются:

- реализация безопасного хранения паролей
- применение JWT для авторизации
- разграничение ролей
- учёт товаров по размерам и остаткам
- транзакционное оформление заказа
- модульная архитектура сервера
- полноценный пользовательский и административный интерфейс

Эти механизмы показывают, что проект выходит за рамки демонстрационного каталога и представляет собой законченную учебную информационную систему.

						КР-09.02.07-3-44-26ПЗ	Лист
							46
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

ЗАКЛЮЧЕНИЕ

В ходе практической реализации была разработана серверная веб-система интернет-магазина одежды, построенная на основе Node.js, Express, EJS и MySQL. Приложение обладает модульной архитектурой, включает пользовательскую и административную части, использует реляционную базу данных и реализует основные сценарии электронной торговли.

В системе реализованы регистрация, аутентификация, каталог товаров, карточка товара, корзина, оформление заказа, личный кабинет пользователя и административная панель. Особенностью проекта является учёт размеров и остатков товаров, что делает модель данных более приближенной к реальной предметной области магазина одежды.

В процессе разработки была сформирована структура проекта, включающая маршруты, контроллеры, middleware, представления и вспомогательные модули. Такое разделение позволило обеспечить логичную организацию кода и упростить дальнейшее сопровождение приложения. Каждый функциональный раздел системы вынесен в отдельные файлы, что делает проект более понятным и расширяемым.

Особое внимание было уделено работе с базой данных. Были спроектированы таблицы для хранения пользователей, товаров, размеров, корзины, заказов и состава заказов. Между таблицами установлены логические связи, обеспечивающие целостность данных. За счёт использования отдельных таблиц для размеров и остатков удалось реализовать более точную модель предметной области, характерную именно для интернет-магазина одежды.

Также в проекте реализованы механизмы защиты пользовательских данных. Пароли пользователей не сохраняются в открытом виде, а хешируются перед записью в базу данных. Для авторизации применяется JWT-токен, который позволяет определять текущего пользователя при

						КР-09.02.07-3-44-26ПЗ	Лист
							47
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

последующих запросах. Разграничение прав доступа обеспечивает разделение возможностей обычного пользователя и администратора.

Практическая реализация показала, что выбранный технологический стек подходит для создания подобных веб-приложений. Node.js и Express обеспечивают удобную серверную обработку запросов, EJS позволяет формировать динамические HTML-страницы, а MySQL предоставляет надёжное хранение структурированных данных.

Разработанная система может быть использована как основа для дальнейшего развития. В перспективе проект можно дополнить онлайн-оплатой, системой отзывов, избранными товарами, уведомлениями о заказах, расширенной фильтрацией, поиском по нескольким параметрам и более детальной административной аналитикой.

Реализованные механизмы безопасности, валидации данных, разграничения доступа и транзакционного оформления заказа позволяют сделать вывод о том, что разработанный проект представляет собой полноценный учебный пример веб-приложения.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Гамма, Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. — Санкт-Петербург : Питер, 2021.
2. Фримен, Э. Изучаем программирование на JavaScript / Э. Фримен, Э. Робсон. — Москва : Питер, 2022.
3. Флэнаган, Д. JavaScript. Подробное руководство / Д. Флэнаган. — Санкт-Петербург : Символ-Плюс, 2023.
4. Стефанов, С. Node.js. Руководство разработчика / С. Стефанов. — Москва : Вильямс, 2022.
5. Хантер, Д. Разработка веб-приложений с использованием Express.js / Д. Хантер. — Санкт-Петербург : Питер, 2021.
6. Роббинс, Д. HTML5 и CSS3. Разработка веб-сайтов / Д. Роббинс. — Санкт-Петербург : Питер, 2022.
7. Кейн, К. MySQL. Руководство администратора / К. Кейн, Дж. Патрик. — Москва : Диалектика, 2021.
8. Шилдт, Г. Полный справочник по SQL / Г. Шилдт. — Москва : Вильямс, 2020.
9. OWASP Foundation. OWASP Top 10 — 2023: основные угрозы веб-приложений [Электронный ресурс]. — <https://owasp.org>
10. Mozilla Developer Network. Web Docs: HTML, CSS, JavaScript [Электронный ресурс]. — <https://developer.mozilla.org>
11. Node.js Foundation. Node.js Documentation [Электронный ресурс]. — <https://nodejs.org>
12. Express.js. Official Documentation [Электронный ресурс]. — <https://expressjs.com>
13. MySQL. MySQL 8.0 Reference Manual [Электронный ресурс]. — Режим <https://dev.mysql.com/doc>
14. W3C. HTML Living Standard [Электронный ресурс]. — <https://html.spec.whatwg.org>
15. Fielding, R. Architectural Styles and the Design of Network-based Software Architectures : dissertation / R. Fielding. — Irvine, 2000.

Er-диаграмма базы данных

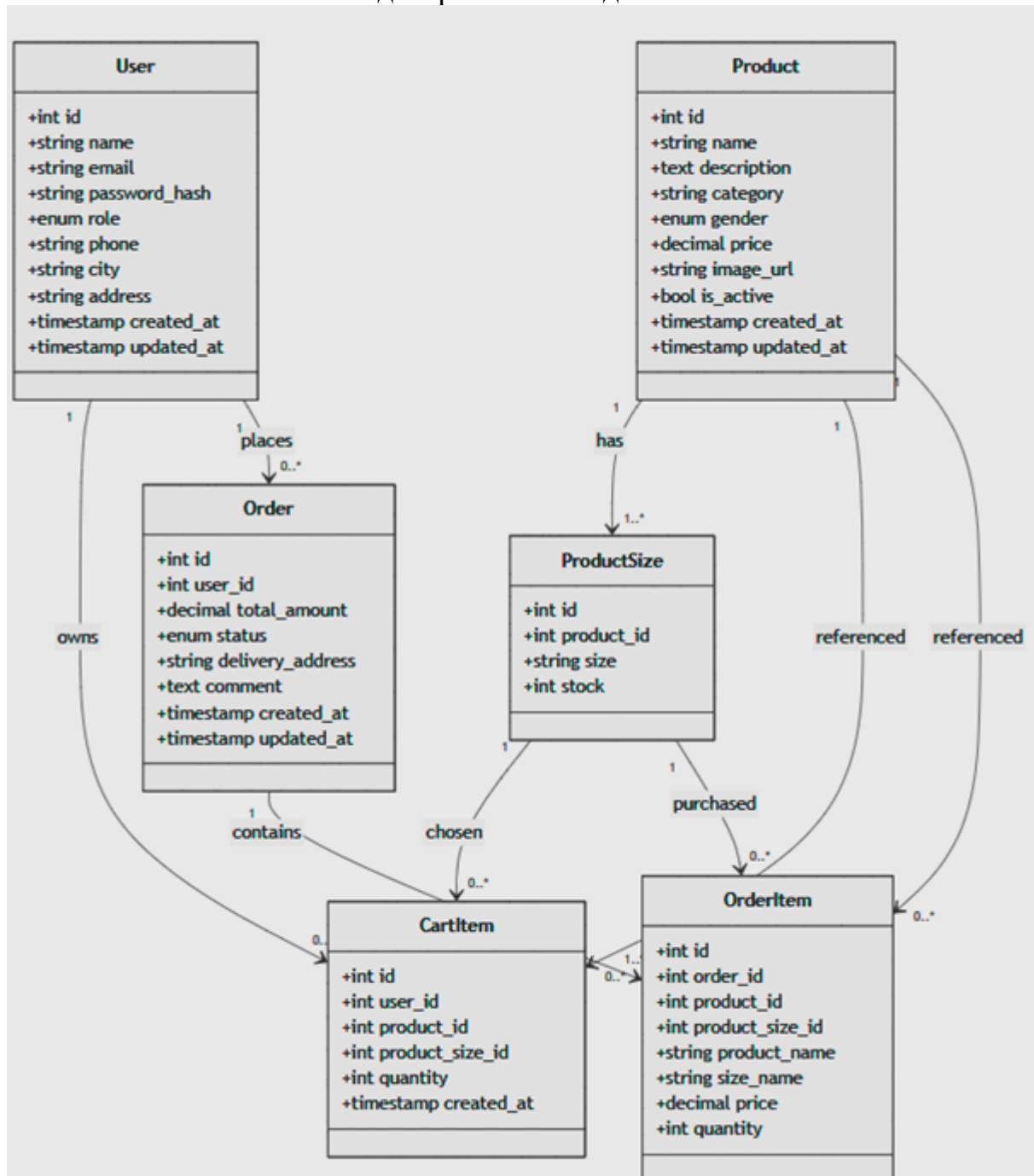
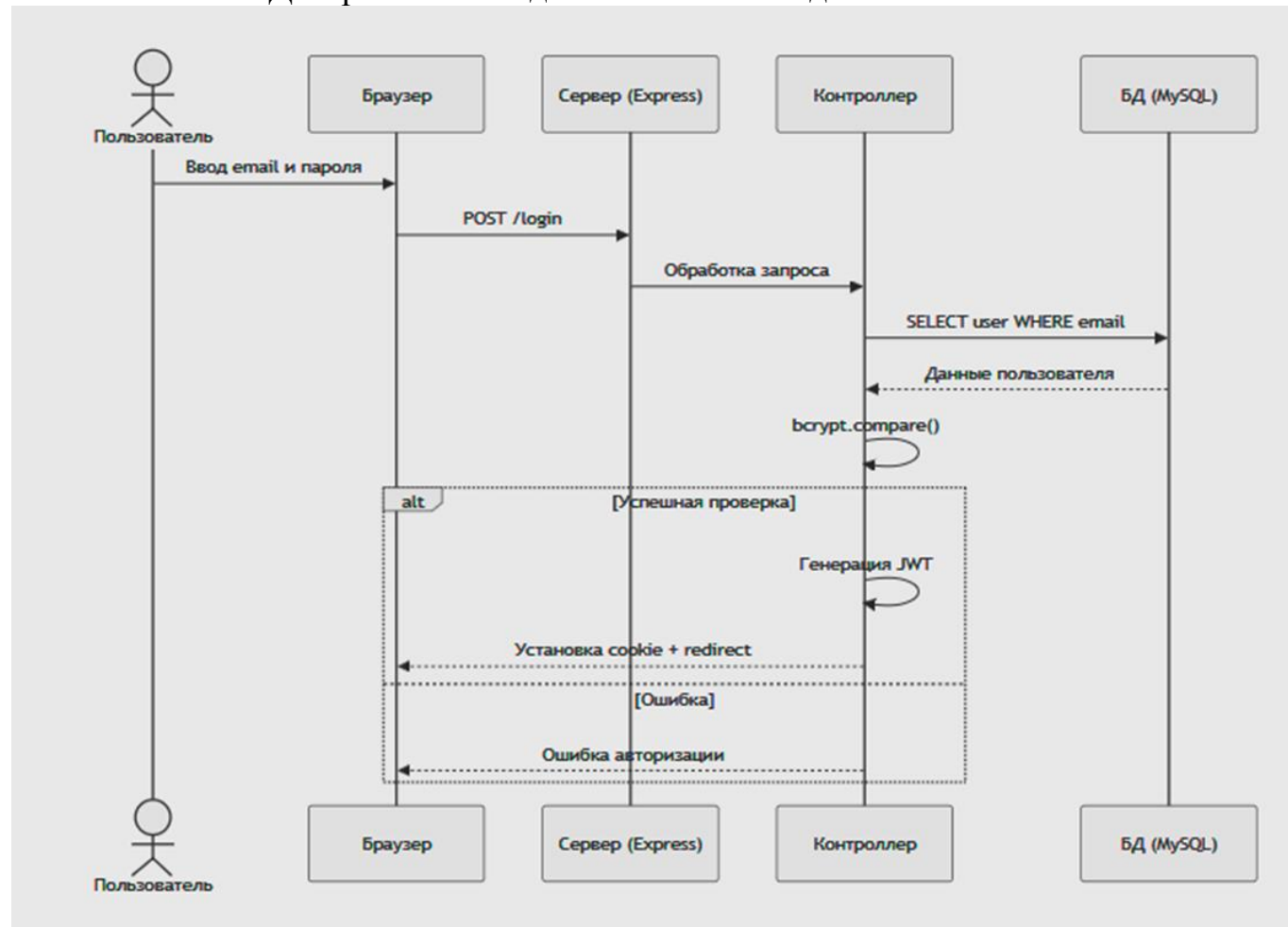


Диаграмма последовательности входа пользователя



ПРИЛОЖЕНИЕ В

Диаграмма последовательности для просмотра каталога товаров пользователем

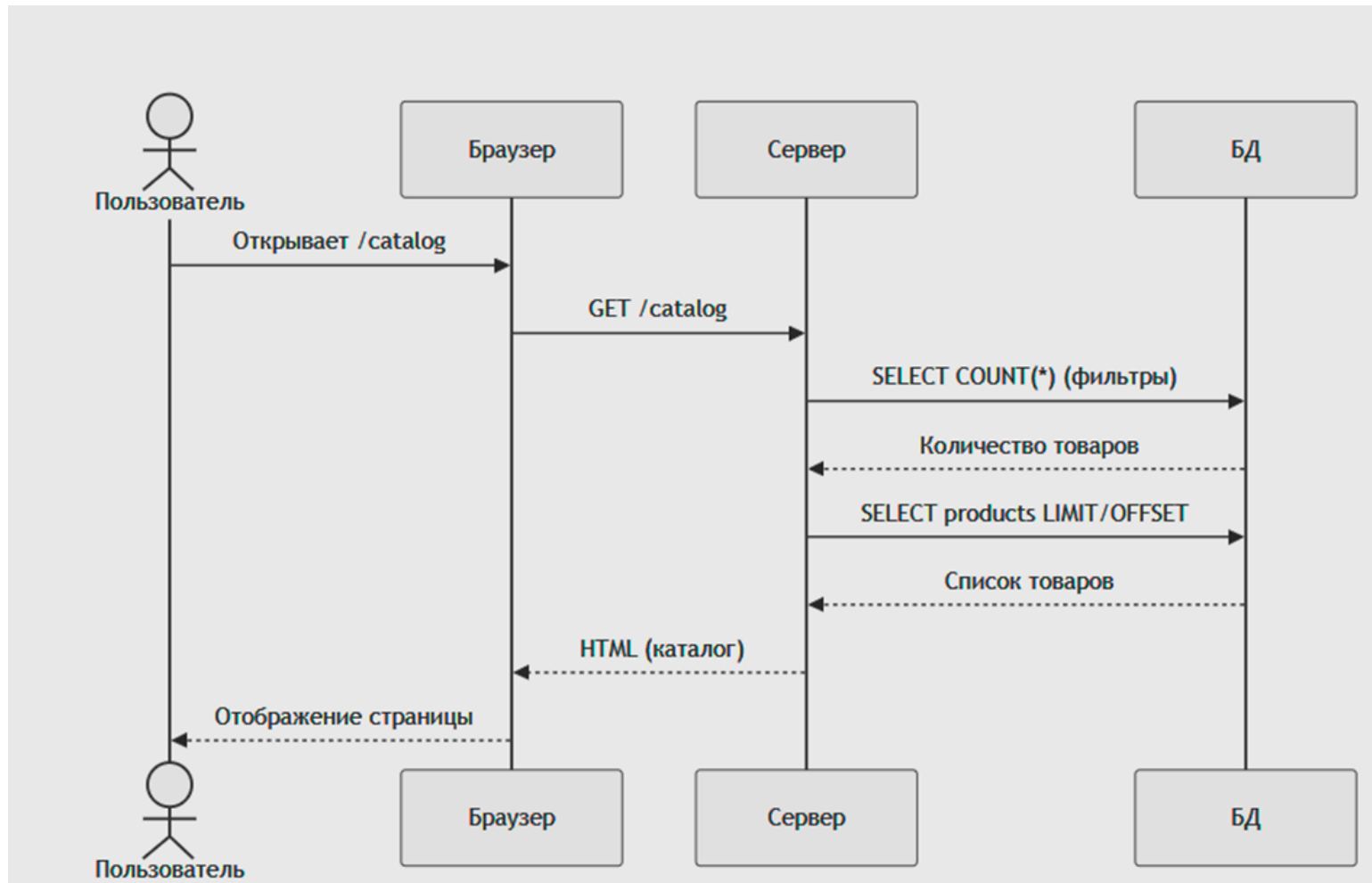
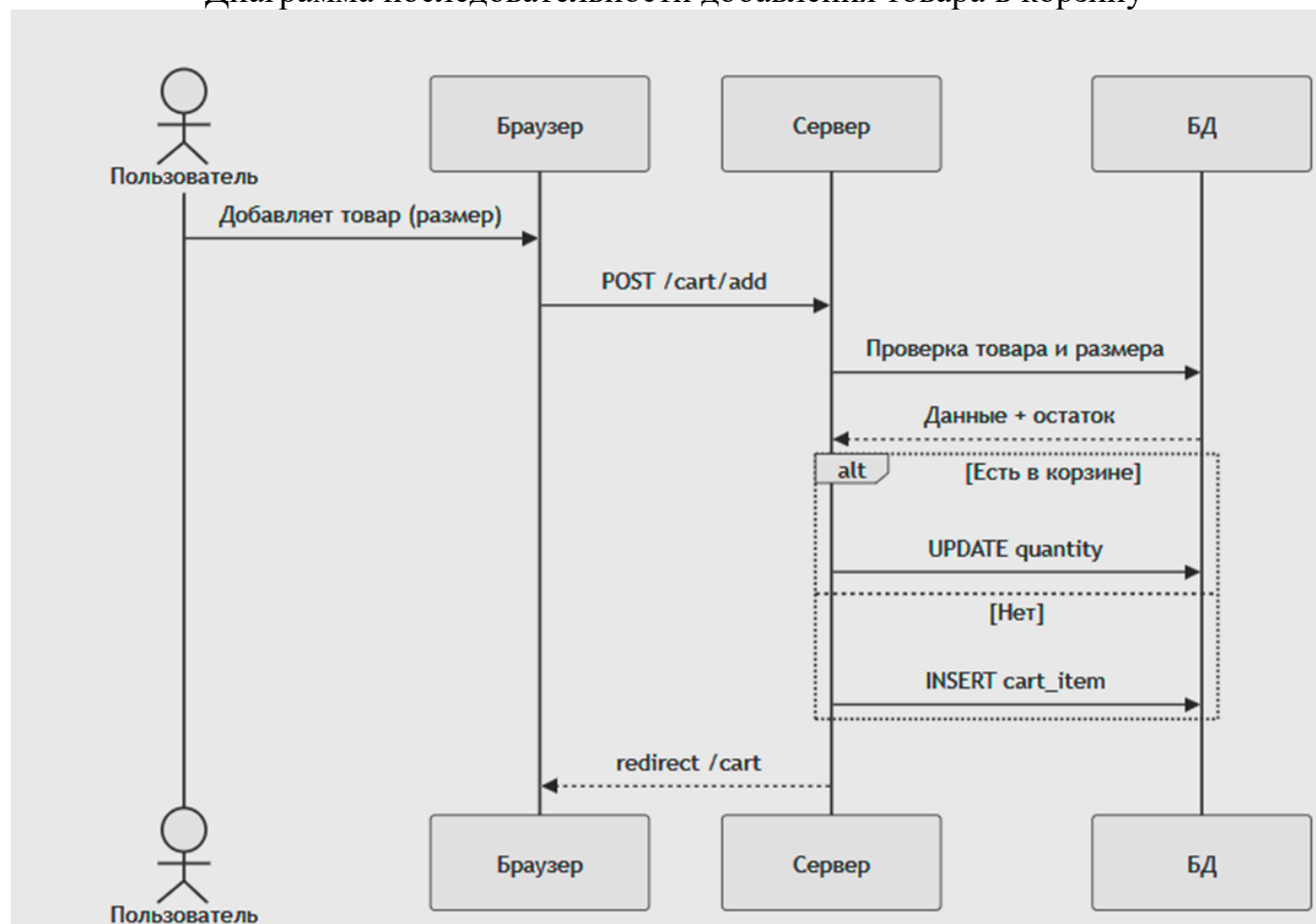


Диаграмма последовательности добавления товара в корзину



ПРИЛОЖЕНИЕ Д

Диаграмма последовательности добавления нового товара администратором

